



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**FinanceFlow: Plataforma Web de Gestión
Financiera e Inteligencia Artificial para el Control de
Gastos Personales**

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Sebastian Arce Bracamonte

(2019062886)

Brant Antony Chata Choque

(2020067577)

Tacna – Perú

2026

Sistema FinanceFlow

Documento Informe de Proyecto utilizando metodología ágil

SCRUM

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	SRAB	BACC	BACC	28/06/2026	Versión Original

INDICE GENERAL

Objetivos: 5

1. Introducción. 6

- 1.1. Propósito del documento. 6
- 1.2. Alcance del proyecto. 6
- 1.3. Audiencia. 6
- 1.4. Estructura del documento. 6

2. Visión. 6

- 2.1. Resumen del producto. 6
- 2.2. Principales características del producto. 6
- 2.3. Objetivos del negocio. 6

3. Organización y roles. 6

- 3.1. Equipo SCRUM... 6
 - 3.1.1. Product Owner. 6
 - 3.1.2. Scrum Master. 6
 - 3.1.3. Equipo de Desarrollo. 7
- 3.2. Stakeholders. 7

4. Backlog del Producto. 7

- 4.1. Definición del Product Backlog. 7
- 4.2. Historias de usuario clave. 7
 - 4.2.1. Historia de usuario 1. 7
 - 4.2.2. Historia de usuario 2. 7
- 4.3. Priorización de las historias de usuario. 7

5. Planificación de Sprints. 7

- 5.1. Ciclo de vida de los sprints. 7
- 5.2. Objetivos de Sprint. 7
- 5.3. Sprint Backlog. 8

5.4.	<i>Reuniones clave del sprint.</i>	8
5.4.1.	<i>Planificación del Sprint (Sprint Planning).</i>	8
5.4.2.	<i>Daily Standup (Daily Scrum).</i>	8
5.4.3.	<i>Revisión del Sprint (Sprint Review).</i>	8
5.4.4.	<i>Retrospectiva del Sprint (Sprint Retrospective).</i>	8
6.	Definición de "Hecho" (Definition of Done - DoD).	8
6.1.	Criterios de aceptación.	8
6.2.	Requisitos de calidad.	8
7.	Gestión de Impedimentos.	9
7.1.	<i>Proceso para identificar y resolver impedimentos.</i>	9
7.2.	<i>Escalación de problemas.</i>	9
8.	Iteraciones.	9
8.2.	<i>Revisión de entregas.</i>	9
9.	Métricas y seguimiento de progreso.	9
9.1.	<i>Burndown Chart.</i>	9
9.2.	<i>Velocidad del equipo.</i>	9
9.3.	<i>Revisión de progreso.</i>	9
10.	Plan de Pruebas y control de calidad.	10
10.1.	<i>Pruebas en el marco de SCRUM...</i>	10
10.2.	<i>Automatización de pruebas.</i>	10
10.3.	<i>Criterios de aceptación de historias.</i>	10
11.	Riesgos y Gestión de cambios.	10
11.1.	<i>Riesgos del proyecto.</i>	10
11.2.	<i>Gestión del cambio.</i>	10
12.	Conclusiones.	10
12.1.	<i>Reflexiones finales.</i>	10
12.2.	<i>Próximos pasos.</i>	10

Informe de Proyecto utilizando metodología ágil SCRUM

La documentación de un proyecto de software utilizando la metodología ágil SCRUM debe ser ligera, enfocada en la colaboración y la entrega continua de valor. A diferencia de las metodologías tradicionales, SCRUM promueve la iteración rápida y la flexibilidad, por lo que la documentación tiende a ser más adaptativa y funcional.

Sin embargo, a pesar de la flexibilidad, es importante que haya documentación que guíe el proyecto y facilite la toma de decisiones, especialmente para garantizar la trazabilidad, la calidad y el cumplimiento de los requisitos. A continuación, se plantea una estructura recomendada para un documento que pueda ser utilizado en un proyecto ágil SCRUM.

Documentación ligera: SCRUM promueve la documentación mínima necesaria, enfocándose más en la comunicación y la colaboración entre los miembros del equipo y stakeholders. Por lo tanto, la documentación debe ser accesible, pero sin ser redundante.

Iteración continua: Como SCRUM es un proceso iterativo, la documentación puede ser actualizada de manera continua y evolutiva según el avance del proyecto.

Esta estructura ayuda a mantener el proyecto ágil, enfocado en la entrega continua de valor y adaptado a los cambios que puedan surgir durante el ciclo de vida del producto.

Objetivos:

El objetivo principal de este documento es establecer y formalizar el marco de trabajo ágil basado en SCRUM para el desarrollo, seguimiento y entrega continua del sistema FinanceFlow. Busca estandarizar los procesos del equipo de desarrollo, asegurar la calidad del producto final a través de prácticas de ingeniería consolidadas y mantener una alineación constante entre los requerimientos del negocio y la implementación técnica. Este enfoque iterativo reemplaza cualquier metodología tradicional previa (como RUP), priorizando la adaptabilidad y la entrega temprana de valor.

1. Introducción

1.1. Propósito del documento

El propósito de este documento es definir detalladamente la metodología SCRUM aplicada en el ciclo de vida del proyecto FinanceFlow. Sirve como la principal guía operativa para el equipo, detallando cómo se estructuran los roles, las ceremonias obligatorias, los artefactos generados y los criterios de calidad técnica (Definition of Done) que rigen cada entrega.

1.2. Alcance del proyecto

FinanceFlow es un sistema web (con planes de expansión a ecosistemas móviles vía React Native) dedicado a la gestión financiera personal. El alcance técnico abarca:

- Una arquitectura cliente-servidor robusta y asíncrona.
- Autenticación segura y flujos de recuperación de contraseñas.
- Integración de servicios avanzados de Inteligencia Artificial (Machine Learning, OCR para comprobantes y NLP para categorización).
- Un ecosistema de reportes y planificación de compras altamente interactivo.
- Módulos administrativos para cierres mensuales y gestión de caja chica.

1.3. Audiencia

Este documento está redactado de manera exhaustiva para múltiples perfiles:

- Equipo de Desarrollo y Arquitectura: (Sebastian Arce, Brant Chata) para tener un estándar claro sobre cómo codificar, probar y entregar.
- Stakeholders y Patrocinadores: (Mag. Ricardo Valcárcel) para comprender cómo se gestiona el riesgo y cómo se asegura el progreso de su inversión o requerimiento académico.
- Auditores de Calidad o Nuevos Colaboradores: Como material de onboarding para entender rápidamente la madurez de los procesos del equipo.

1.4. Estructura del documento

El documento se divide en 12 áreas clave que abarcan desde la visión estratégica del producto, pasando por la organización del equipo (roles), la gestión del Backlog (requerimientos), las reglas que rigen las

iteraciones (Sprints), los estrictos criterios de aceptación de código (DoD), hasta la gestión proactiva de riesgos y el control de calidad.

2. Visión

2.1. Resumen del producto

FinanceFlow trasciende la idea de una simple "hoja de cálculo web". Se concibe como una plataforma integral, predictiva y automatizada que asiste a los usuarios en el control exhaustivo de sus finanzas personales. Al minimizar el esfuerzo manual (a través de la IA) y maximizar el entendimiento visual de los datos, el sistema busca convertirse en un hábito diario para la salud financiera del usuario.

2.2. Principales características del producto

- Dashboard Analítico en Tiempo Real: Implementado con la librería Recharts, ofrece visualizaciones dinámicas de ingresos, egresos y saldos netos.
- Categorización Inteligente y Automatizada: Uso de un microservicio de Machine Learning en Python (Multinomial Naive Bayes y pipeline TF-IDF) para clasificar gastos basándose en la descripción ingresada.
- Procesamiento de Comprobantes (OCR): Extracción automatizada de montos y fechas a partir de imágenes (tickets o capturas de pantalla de Yape/Plin), reduciendo la fricción del ingreso de datos.
- Gestión Avanzada de Periodos: Planificador de compras independiente, cierres mensuales automáticos (con validaciones lógicas) y gestión de caja chica.
- Arquitectura de Alta Tolerancia a Fallos: Implementación de un SDK wrapper en el frontend que permite el retroceso automático (fallback) a métodos tradicionales si la red o los servicios modernos fallan.

2.3. Objetivos del negocio

- Adopción y Retención: Proveer una experiencia de usuario (UX) tan fluida que reduzca la tasa de abandono típica en las aplicaciones de finanzas.
- Prevención Financiera: Ayudar activamente a los usuarios a evitar el sobreendeudamiento mediante alertas inteligentes y planificación predictiva.
- Sostenibilidad Operativa: Mantener una arquitectura Serverless y optimizada (Dockerfiles ajustados para evitar Out of Memory)

que permita operar el sistema en capas gratuitas (Free Tiers) durante su etapa inicial.

3. Organización y roles

En SCRUM, la correcta delimitación de roles es vital. Dado el contexto de un equipo compacto, los roles asumen un formato pragmático:

3.1. Equipo SCRUM

3.1.1. Product Owner

Responsable de maximizar el valor del producto y del trabajo del equipo de desarrollo.

- Funciones aplicadas: Es el único perfil con autoridad para reordenar el Product Backlog. En el contexto de FinanceFlow, este rol recaba las expectativas de la cátedra/mercado y las traduce en Requerimientos Funcionales concretos. Se asegura de que historias como "el cierre mensual" tengan precedencia lógica sobre "mejoras estéticas".

3.1.2. Scrum Master

Actúa como un líder servicial y facilitador.

- Funciones aplicadas: Rol rotativo entre Sebastian Arce y Brant Chata. Su tarea crítica no es asignar trabajo, sino remover bloqueos técnicos. Por ejemplo, si el equipo enfrenta problemas de CORS con Vercel o límites de memoria en Render, el Scrum Master asume la tarea de desbloquear (ej. commits como fix: configurar CORS dinámico).

3.1.3. Equipo de Desarrollo

Integrantes fijos: Sebastian Arce y Brant Chata.

- Funciones aplicadas: Un escuadrón auto-gestionado y multidisciplinario (Full-Stack). No existen silos; ambos desarrollan interfaces en React, APIs en Node.js, y modelos predictivos en Python. Ellos tienen la última palabra sobre cuánto trabajo pueden comprometerse a realizar en un Sprint.

3.2. Stakeholders

Mag. Ricardo Valcárcel (y panel evaluador): Actúa como el cliente final y validador de los objetivos de negocio. Proporciona el feedback

necesario durante las ceremonias de Sprint Review para asegurar que el incremento de software cumpla con los estándares esperados.

4. Backlog del Producto

4.1. Definición del Product Backlog

Es la fuente única de verdad para cualquier cambio a realizarse en el sistema. Consiste en 15 Requerimientos Funcionales (RF) documentados que evolucionan iterativamente. Las tareas puramente técnicas (como refactorizar el SDK o crear Dockerfiles) también viven aquí y compiten en prioridad con las funcionalidades de negocio.

4.2. Historias de usuario clave

4.2.1. Historia de usuario 1: Cierre Mensual y Caja Chica

- Título: Ejecución de Cierre Contable Mensual
- Descripción: Como usuario activo del sistema, quiero poder realizar un "cierre de mes" para bloquear ediciones pasadas, archivar mis movimientos históricos y trasladar mis saldos netos como saldo inicial (caja chica) del siguiente periodo.
- Criterios de validación: El sistema debe impedir ingresos de fechas en un mes cerrado. Debe generar un reporte inmutable.

4.2.2. Historia de usuario 2: Recuperación Segura de Acceso

- Título: Recuperación de Contraseña con Brevo API
- Descripción: Como usuario que ha olvidado su clave, quiero ingresar mi correo y recibir un enlace cifrado de uso único y tiempo limitado, para poder definir una nueva contraseña de forma segura y sin intervención de soporte técnico.
- Criterios de validación: El enlace debe expirar. El sistema debe implementar un "modo demo" (fallback) si el proveedor SMTP (Brevo) falla por timeouts.

4.3. Priorización de las historias de usuario

La priorización en FinanceFlow ha seguido una estrategia de "Risk-First" (Riesgo Primero) y "Architecture-First" (Arquitectura Primero):

1. Sprints Iniciales: Autenticación base, estructura de base de datos y migración al SDK (para mitigar el riesgo de deuda técnica).

2. Sprints Intermedios: Integración de Inteligencia Artificial (mayor riesgo tecnológico) e integración de correos reales.
3. Sprints Finales: Consolidación de flujos de valor directo al usuario final (Planificador, Cierres mensuales y mejoras de UI con Recharts).

5. Planificación de Sprints

5.1. *Ciclo de vida de los sprints*

El desarrollo de FinanceFlow documenta el paso a través de 6 Sprints. Adaptándose a un entorno de desarrollo ágil, las entregas se agruparon en clusters de valor. En lugar de desplegar pedazos incompletos cada viernes, el equipo agrupó características lógicas para desplegarlas en bloques sólidos (ej. un sprint enfocado puramente en ML en abril, otro en la recuperación de cuentas a finales de mayo, y los flujos finales a mediados de junio).

5.2. *Objetivos de Sprint*

Cada Sprint se guió por una meta innegociable:

- Sprint 1-2: "Sistema Core y Arquitectura Resiliente" (Implementación de SDK y CRUD básico).
- Sprint 3-4: "Inteligencia Artificial y Seguridad" (Modelos Naive Bayes y flujos SMTP).
- Sprint 5-6: "Experiencia de Usuario y Finalización de Ciclo Financiero" (Cierres, planificadores independientes y borrados lógicos).

5.3. *Sprint Backlog*

Para cada iteración, el Equipo de Desarrollo seleccionó un subconjunto del Product Backlog, lo desglosó en tareas técnicas puntuales (commits como feat:, fix:, chore:) y se responsabilizó colectivamente de su finalización.

5.4. *Reuniones clave del sprint*

5.4.1. *Planificación del Sprint (Sprint Planning)*

Ceremonia de máximo 4 horas donde el Product Owner explica la prioridad y el Equipo de Desarrollo diseña la solución técnica. Aquí se define el Sprint Goal.

5.4.2. *Daily Standup (Daily Scrum)*

Sincronizaciones rápidas para inspeccionar el avance. Dado el equipo pequeño, estas sesiones a menudo resultan en sesiones de pair programming (programación en pareja) instantáneas si se detecta un bloqueo técnico severo.

5.4.3. *Revisión del Sprint (Sprint Review)*

Al finalizar el Sprint, no se muestran diapositivas, se muestra software funcionando. El equipo demuestra las funcionalidades desplegadas en los entornos reales (Vercel/Render) y el Stakeholder (Mag. Ricardo Valcárcel) brinda retroalimentación.

5.4.4. *Retrospectiva del Sprint (Sprint Retrospective)*

El espacio de mejora técnica. Fue producto de estas reuniones que el equipo decidió refactorizar fuertemente su código y crear el patrón de Adaptadores en el frontend, luego de notar que las llamadas directas a la API eran frágiles y difíciles de mantener.

6. Definición de "Hecho" (Definition of Done - DoD)

En FinanceFlow, un Requerimiento Funcional no está "hecho" simplemente porque funciona en la computadora del desarrollador ("It works on my machine").

6.1. Criterios de aceptación

La historia cumple fielmente con lo solicitado por el usuario y maneja los casos límite (ej. ¿Qué pasa si el OCR lee una imagen sin símbolo de moneda? El sistema debe aplicar el fallback avanzado implementado en el commit e9efc81).

6.2. Requisitos de calidad

Para que un código sea integrado a la rama main, debe cumplir un patrón arquitectónico comprobable en el repositorio:

1. Estructura Backend:

- La nueva ruta está correctamente registrada en backend/routes/.
- Se aplican los middlewares de seguridad correspondientes (auth.js para rutas privadas).

- La lógica está separada: el Controlador (backend/controllers/) solo maneja la petición/respuesta, y la lógica de negocio vive aislada en el Servicio (backend/services/).
2. Estructura Frontend:
 - No se hacen llamadas directas con axios desde los componentes. Todo pasa por la capa de services/ (ej. usuarios-adapter.js).
 3. Pruebas y Calidad:
 - El código backend y el SDK deben mantener intacta la suite de 94 pruebas automatizadas (100% de éxito).
 4. Despliegue Funcional (Potentially Shippable Increment):
 - El código empujado a main debe compilar sin errores en los pipelines automatizados de Render y Vercel.

7. Gestión de Impedimentos

7.1. Proceso para identificar y resolver impedimentos

El equipo practica una filosofía de Fail Fast (fallar rápido). Si un servicio externo (como una pasarela SMTP o la base de datos de MongoDB) falla, la aplicación está diseñada para manejarlo elegantemente. Técnicamente, el repositorio está plagado de comentarios // Fallback a método original que demuestran la preparación ante impedimentos de red.

7.2. Escalación de problemas

Cuando surge un problema bloqueante en producción, se abandona temporalmente el Sprint Backlog para aplicar un Swarming (enjambre). Toda la capacidad del equipo se enfoca en resolver la caída. Esto ha quedado documentado en la historia del proyecto mediante commits de alta prioridad:

- EMERGENCY FIX: Reemplazar servicios con versión de emergencia
- HOTFIX: Corregir errores de runtime en adaptadores

8. Iteraciones

8.1. Incrementos del producto

Las iteraciones en SCRUM permiten que la arquitectura madure de manera orgánica. A lo largo del proyecto, FinanceFlow evolucionó de tener lógica acoplada a desarrollar un SDK Wrapper (sdk/) totalmente independiente, abriendo las puertas para que el día de mañana se integre el cliente React Native (chore: scaffold inicial para cliente React Native).

8.2. *Revisión de entregas*

La revisión de entregas no es un proceso estático. El repositorio hace uso del despliegue continuo vinculado a la nube (Continuous Deployment). Acciones como FORCE VERCEL: Trigger new deployment evidencian que el repositorio está conectado directamente a la infraestructura, de manera que la rama main refleja siempre la versión más estable de producción.

9. **Métricas y seguimiento de progreso**

9.1. *Burndown Chart*

El equipo realiza seguimiento constante de la reducción de historias de usuario. Los 15 Requerimientos Funcionales (RF) son quemados sprint a sprint. Si un requerimiento resulta más complejo (como el procesamiento NLP), el progreso se visualiza en cómo se van cerrando los sub-tasks.

9.2. *Velocidad del equipo*

Analizando el historial de versiones (Git Commit Patterns), la velocidad del equipo experimentó picos lógicos. El equipo fue capaz de manejar altas ráfagas de productividad (10 a 15 commits complejos diarios) en las fases clave de integración, demostrando que la base de código inicial era lo suficientemente limpia para permitir un desarrollo acelerado posterior.

9.3. *Revisión de progreso*

A diferencia de proyectos tradicionales, el progreso no solo se mide por "ventanas nuevas". El equipo asignó tiempo valioso a la reducción de deuda técnica, optimizando el rendimiento (ej. optimizar Dockerfiles para ahorrar memoria, excluir movimientos inactivos de los análisis, etc.).

10. **Plan de Pruebas y control de calidad**

10.1. *Pruebas en el marco de SCRUM*

La calidad no es una fase al final del proyecto, es una actividad continua. El equipo utiliza Test-Driven Development (TDD) y pruebas de regresión como red de seguridad antes de cada entrega importante.

10.2. Automatización de pruebas

- FinanceFlow posee una suite de 94 tests automatizados, asegurando que los casos críticos no se rompan por accidente.
- Se utilizan frameworks estándar de la industria: Jest para las aserciones y lógicas puras del backend/SDK, y React Testing Library para asegurar que el DOM del frontend se comporte como se espera.
- Oportunidades de mejora detectadas en el código: Actualmente, el control es manual. Si bien la suite existe, los desarrolladores dependen de su disciplina para ejecutar npm test. Una próxima iteración de madurez SCRUM exigiría configurar Git Hooks (como Husky) o GitHub Actions para que los bloqueos sean mecánicos.

10.3. Criterios de aceptación de historias

El código es honesto sobre su estado. A través de inspecciones del repositorio, se ha documentado la existencia de múltiples marcas TODO: Implementar test de inicialización o TODO: Implementar test completo en archivos como movimientos-adapter.test.js. Esto demuestra un proceso transparente de seguimiento de deuda técnica que deberá ser resuelto en los siguientes Sprints de mantenimiento.

11. Riesgos y Gestión de cambios

11.1. Riesgos del proyecto

A través del ciclo de vida ágil, se identificaron y gestionaron riesgos técnicos formidables:

- Riesgo de Infraestructura (Capa Gratuita): Render genera "Cold Starts" (tiempos largos de encendido) y la base MongoDB M0 tiene un límite severo de 512MB.
 - Mitigación: Se optimizaron las dependencias (ej. Remover dependencias obsoletas de video GLX en Debian Trixie (usamos OpenCV Headless)) y los Dockerfiles para evitar caídas por falta de memoria.
- Riesgos de Seguridad:
 - Identificado: Manejo de JWT de forma stateless (sin revocación en tiempo real).
 - Identificado: Fugas potenciales de credenciales.
 - Mitigación: El equipo reaccionó rápido eliminando credenciales reales de los archivos .env.example y documentando estrictamente que el sistema debe funcionar bajo variables seguras inyectadas en la nube.
- Riesgos de Fiabilidad (IA): Los sistemas de OCR y NLP son probabilísticos.

- Mitigación: Se incluyó el Reentrenamiento Continuo (Continuous Learning / Human-In-The-Loop) permitiendo a los usuarios corregir a la IA para mejorar su precisión con el tiempo.

11.2. Gestión del cambio

SCRUM abraza el cambio. En FinanceFlow, la arquitectura de Adaptadores (Adapter Pattern) permite que si un requisito tecnológico cambia abruptamente, la interfaz gráfica no sufra daños. Si se decide abandonar el SDK actual por uno nuevo, el impacto está confinado al servicio intermedio, protegiendo así la inversión en tiempo y desarrollo visual.

12. Conclusiones

12.1. Reflexiones finales

La implementación de la metodología SCRUM ha sido el factor diferencial para que FinanceFlow pase de ser un concepto a un producto robusto, tolerante a fallos y con características de vanguardia (AI). La estructura iterativa, sumada a la disciplina del equipo para atacar rápidamente los problemas críticos (Hotfixes) y su compromiso con la calidad arquitectónica (DoD estricto, 94 tests, SDK wrappers), ha resultado en una solución financiera con proyección profesional.

12.2. Próximos pasos

Para continuar elevando los estándares de ingeniería y gestión del proyecto, se establecen las siguientes prioridades a corto plazo:

Integración y Entrega Continua (CI/CD): Implementar .github/workflows para automatizar la ejecución de las 94 pruebas antes de permitir cualquier Merge a la rama principal.

Cobertura de Pruebas del SDK: Abordar el backlog técnico enfocado en resolver los comentarios TODO de los adaptadores, asegurando la fiabilidad 100% de los nuevos flujos.

Desarrollo Móvil: Aprovechar el terreno preparado (chore: scaffold inicial para cliente React Native) para planificar Sprints puramente enfocados en el lanzamiento de la app móvil en tiendas.